VERIQO -- AUTHORITY TO OS INTEGRATION AUDIT: GAP ANALYSIS & ROADMAP

Response to Mengubah_fokus_VERIQO_sekarang.docx's reframing: not "is there one more setter that can be bypassed" but "can the hardened authority model survive integration with the real VERIQO operating system." The reviewer proposed the next major audit as VERIQO AUTHORITY -> OS INTEGRATION AUDIT, across ten boundaries (A-J) and one end-to-end adversarial pipeline scenario.

This document is that audit's gap analysis and roadmap, not its execution. Executing ten boundary audits plus a full end-to-end adversarial pipeline test is realistically several more engineering rounds of work -- the reviewer's own docx frames it as "the next major audit," future tense, not something to complete in one pass. What this round delivers instead, honestly and completely: a real, grep-and-read inventory of what currently exists for each boundary, whether it is wired to the hardened authority core at all, and a concrete, prioritized list of what remains to be built before each boundary can actually be audited (because for most of them, the integration doesn't exist yet to audit).

Every claim below is grounded in a real grep/file read against this repository as of this round; nothing here is inferred from the package name alone.

PART 1 -- THE HEADLINE FINDING

Six-plus rounds of hardening work proved the Evidence/Manifest/Hypothesis/Finding authority packages safe in isolation. Almost none of VERIQO's surrounding OS-layer packages import those hardened packages at all today. A repository-wide search for every OS-layer package importing any of pkg/evidence/manifest, pkg/insurance/evidence, pkg/insurance/causation, pkg/insurance/finding, pkg/insurance/cre, or pkg/evidence/provenance found exactly two real hits outside the authority packages' own internal cross-references:

1. pkg/insurance/dossier/dossier.go:15 imports pkg/insurance/causation -- and only to embed an already-computed *causation.Explanation into a report struct (dossier.go:37,78). Read-only consumption of a derived value; no construction, no authority risk.
2. pkg/consensus/raftlite/fsm_manifest_adapter.go and fsm_evidence_adapter.go -- the FSM adapters this round's earlier work (L99 Gap Coverage Response) just built, wiring real raft consensus to manifest.Registry/evidence.Registry through their own gated methods.

Every other conceptual layer the reviewer named -- API, Workflow, Knowledge, Intelligence, Decision, Ledger, Storage (beyond raftlite's own in-process snapshot mechanism), Replay, and whatever "UCR"/"UER" map onto in this codebase -- has zero import-level connection to the authority core today. This is not a subtle finding requiring careful interpretation: it is a grep returning nothing, repeated ten times.

This is, in one sense, good news: none of these layers can currently bypass the hardened authority packages, because none of them touch the authority packages at all. It is also the reviewer's exact point: there

is no "system-level authority assurance" to claim yet, because there is close to no system-level integration to audit. The boundaries A-J are not yet safe OR unsafe -- they are not yet built, for the most part.

PART 2 -- BOUNDARY-BY-BOUNDARY INVENTORY

Boundary	Real package(s) found	Wired to hardened core?	Honest assessment
A. API boundary	veriqo/gateway/rest (server.go, lifecycle_route.go, generated_routes.go)	NOT WIRED -- imports pkg/lifecycle, pkg/platform/audit, pkg/platform/security, veriqo/registry; zero import of manifest/evidence/causation/finding/cre/provenance anywhere in the package	Cannot bypass authority it never touches. Also cannot be said to ENFORCE authority yet -- there is no HTTP route in this repo today that reaches a manifest.Manifest or insevidence.Record at all.
B. Workflow boundary	pkg/workflow/workflow.go	NOT WIRED -- zero import of any hardened package	Same as A: no live bypass risk, no live enforcement either.
C. UCR/UER boundary	pkg/ucr exists and is real -- but it is the Unified Cognitive Reasoning engine (working memory, reasoning graph, causal planner; see pkg/ucr/ucr.go's own package doc comment), not a "Unified Case/Evidence Registry." No package named or clearly serving a "UER" (Unified Evidence Registry) role was found anywhere in this repository (a whole-word search for "UER" across all .go files returned zero hits). The closest real candidates for whatever "UER" is meant to name are pkg/insurance/evidence (Evidence Record, already hardened) and pkg/evidence/api (see below).	pkg/ucr itself: NOT WIRED. pkg/evidence/api/api.go:60 and contract.go:44 DO import pkg/evidence/provenance -- but only to reference two constants (provenance.OriginSynthetic, provenance.SchemaVersionV1); no call to GrantTrust, Register, or construction of a provenance.Entry anywhere in that package.	This is the boundary the reviewer's own naming is honestly ambiguous about, given this codebase's actual package names. Recommend clarifying with the reviewer whether "UER" refers to a package that needs to be BUILT, or to pkg/insurance/evidence/pkg/evidence/api under a different name. Either way: no live authority-bypass risk exists today because the connection is, at most, a shared constant reference.
D. Knowledge boundary	veriqo/core/knowledge/knowledge.go, pkg/moat/kg/kg.go (Knowledge Graph, the same package KGAdapter wires to real raft consensus)	NOT WIRED to the authority core -- zero import of manifest/evidence/causation/finding/cre/provenance in either package	The specific risk named ("Knowledge layer turns derived authority into asserted truth") cannot manifest today because Knowledge has no path to READ authority state in the first place, let alone re-assert it. This boundary is closed by absence, not by a checked gate -- worth building the gate deliberately once the pipe is built, rather than relying on the pipe's current non-existence.
E. Intelligence boundary	veriqo/core/intelligence/intelligence.go, pkg/moat/decision/intelligence.go	NOT WIRED -- zero import of the authority core	Same as D: the specific risk ("model output automatically becomes a trusted finding") cannot manifest because Intelligence has no path to finding.Finding/cre.AuthorizedFinding at all today. cre.Authorize/AuthorizeGrounded remain the only real gate, and nothing calls them from this layer.
F. Decision boundary	pkg/moat/decision/decision.go -- a real, standalone weighted-factor Engine with its own Policy/Decision/DecisionRecord types (decision.go:48-140)	NOT WIRED -- does not take a cre.AuthorizedFinding (or anything from the Finding authority chain) as input anywhere; it is a parallel, independent decision-scoring mechanism	This is the most consequential gap in the whole audit. The reviewer's own pipeline diagram puts Decision directly downstream of Finding/CRE; the real code shows two unrelated systems. Until pkg/moat/decision.Engine (or whatever becomes the real Decision layer) is wired to consume an AuthorizedFinding specifically -- not a raw finding.Finding, not a hand-built score -- "Decision only accepts authorized inputs" cannot be tested, because Decision does not accept Finding-shaped inputs of any kind today.
G. Ledger boundary	pkg/platform/audit/merkle.go + audit.go (the Merkle-anchored audit ledger; matches this engagement's own earlier VTECP Phase 5 "Ledger Merkle + Anchor + Verify		

API" work) NOT WIRED -- zero import of the authority core Same absence-based closure as D/E. A ledger record today cannot cite a manifest.Manifest's ManifestHash or an AuthorizedFinding's AuthorizationHash as its provenance, because nothing connects them. Building that connection, with the SAME "derive, never assert" discipline the authority packages already use, is real future work.

H. Storage boundary pkg/storage/store.go, pkg/storage/snapshot (a generic, content-agnostic Raft snapshot mechanism -- Snapshot{Meta, Data []byte}, checksum-verified only, no domain semantics), pkg/storage/wal, pkg/storage/evidence PARTIALLY WIRED, and this needs a precise distinction: pkg/storage/snapshot.Store itself is STILL not wired to manifest.Registry/evidence.Registry -- it remains a generic byte-blob store. What IS now wired (this round's own earlier work) is a DIFFERENT snapshot mechanism: raftlite.Snapshotter, an interface internal to pkg/consensus/raftlite, which ManifestAdapter/EvidenceAdapter implement directly, bypassing pkg/storage/snapshot entirely. Do not conflate the two. "Snapshot restoration" was closed for the raftlite-internal mechanism (real, tested, fail-closed against forgery). pkg/storage/snapshot.Store's OWN persisted-object restore path -- the literal "Storage boundary" the reviewer named ("persisted objects... restore authority") -- has not been touched and has no wiring to the authority packages at all. This is real, unclosed, unclaimed work.

I. Replay boundary pkg/replay/replay.go (a tamper-evident record/replay/verify certificate system) NOT WIRED to manifest.Registry/evidence.Registry -- pkg/evidence/api imports pkg/replay, but pkg/evidence/api is itself a separate evidence layer from pkg/evidence/manifest, with no import path connecting the two "Deterministic replay becoming an alternate authority path" cannot manifest via pkg/replay today because pkg/replay has no way to write into manifest.Registry/evidence.Registry at all. The manifest package's OWN internal replay-determinism guarantee (TestReplayReproducesIdenticalFinalizedState) is a different, already-closed concern -- proving replaying real gated calls converges safely, not proving pkg/replay's own certificate mechanism can't inject authority, because it never touches the authority packages to test that against.

J. Cluster boundary pkg/transport/rafttcp (real mTLS TCP transport), pkg/consensus/raftlite WIRED, and this is the one boundary with real, tested integration ManifestAdapter/EvidenceAdapter (this round's own earlier work) proved real multi-node raft consensus converges safely and rejects forged snapshots -- but ONLY using raftlite.MemTransport (in-process), not the real rafttcp mTLS transport. The specific risk named ("node-to-node transport becomes an authority injection path") has not been tested against a REAL network transport for these two adapters -- pkg/kernel/distributed's own adapters have been, per test/integration/live_cluster_test.go, but ManifestAdapter/EvidenceAdapter have not yet been run through that same real-transport harness.

PART 3 -- THE END-TO-END ADVERSARIAL PIPELINE

The reviewer's proposed pipeline:

External Input -> API -> UCR -> UER -> Evidence -> Manifest ->
Hypothesis -> Finding -> CRE -> Decision -> Ledger -> Reporting

No test anywhere in this repository exercises this pipeline, or any meaningful fragment of it, end to end. Confirmed by: no file in test/ (test/soak, test/chaos, test/integration, test/stress, test/e2e, test/evidence, test/acceptance) imports both veriqo/gateway/rest and any hardened authority package. test/e2e/eight_blockers -- the closest-sounding candidate by directory name -- tests eight qualification blockers (per TestAllEightBlockersQualifyTogether), not this pipeline. This is expected, not a surprise: Part 2 already showed API/Workflow/Knowledge/Intelligence/Decision/Ledger have no wiring to the authority core for such a test to exercise in the first place.

THE 12 NAMED ATTACKS, HONESTLY SCORED AGAINST WHAT ACTUALLY EXISTS

Attack	Coverage today
1. Forged evidence status	Package-level: closed (evidence.Registry.Submit resets Status; TestSubmitResetsAuthorityBearingFields). End-to-end: no test, no pipe to attack.
2. Forged manifest state	Package-level: closed (RegisterDraft forces State=DRAFT; TestJSONDeserializedManifestCannotManufactureAuthority, TestForgedSnapshotStateFieldCannotBeRestored). End-to-end: no test, no pipe.
3. Forged hash	Package-level: closed (VerifyManifestHash, TestVerifyManifestHashDetectsTampering). End-to-end: no test, no pipe.
4. Forged hypothesis status	Package-level: closed this engagement’s most recent-but-one round (HypothesisSet.Add forces StatusUnproven; TestHypothesisSetAddNeverTrustsACallerAssertedSupportedStatus). End-to-end: no test, no pipe.
5. Forged finding	Package-level: closed (AuthorizedFinding’s unexported, accessor-sealed fields; Authorize/AuthorizeGrounded gates). End-to-end: no test, no pipe -- and Part 2’s Decision-boundary finding means there is nowhere downstream for a forged Finding to even be tested against yet.
6. Forged authority (trust grant)	Package-level: closed (provenance.Registry.Register forces TrustGranted=false; only GrantTrust can set it). End-to-end: no test, no pipe.
7. Forged serialized state	Package-level: closed for both Manifest and Evidence Record (TestJSONDeserializedRecordCannotManufactureAuthority, TestJSONDeserializedManifestCannotManufactureAuthority, and this round’s raftlite Restore forgery tests). End-to-end: no test, no pipe.
8. Replay omission	Package-level: closed (TestReplayOmittingReviewedEventCannotReachFinalized, and the raftlite Restore equivalent). End-to-end: no test, no pipe.
9. Duplicate event	Package-level: closed for concurrent double-finalize/duplicate-transition (TestConcurrentDoubleFinalizeHasExactlyOneWinner, TestConcurrentDuplicateAdvanceTransitionHasExactlyOneWinner). Not separately tested as a REPLAY-time duplicate (the same command applied twice via Apply) for the raftlite adapters specifically -- a real, small, addressable gap.
10. Stale state	Partially covered: RecordCustodyEventDoesNotStaleTheFinalizedManifestHash covers one specific staleness shape. A general "an old cached read is acted on after the authoritative state has moved on" scenario has no dedicated test anywhere in the repo.
11. Concurrent transition	Package-level: closed, seven scenarios (TestConcurrentFinalizeAndCustodyMutateNeverStalesTheHash, TestConcurrentFinalizeAndSupersedeNeverSupersedesAnUnfinalizedParent, TestConcurrentMarkSupersededAndDuplicateSubmitNeverResurrectsEvidence, TestConcurrentSetRightsChangesConvergeToASingleAuthorizedValue, and others from the Trust Authority Model round). End-to-end: no test, no pipe.
12. API injection	No coverage anywhere. veriqo/gateway/rest has no route touching the authority core (Part 2, boundary A), so there is nothing for an "API injection" attack to inject INTO yet. This is the single attack type with zero existing coverage even at the package level, because the API layer itself doesn’t reach the packages that would need defending.

PART 4 -- PRIORITIZED ROADMAP: WHAT STILL NEEDS TO BE BUILT

In the order that unblocks the most subsequent work:

1. Decide what "Decision" means for real, then wire pkg/moat/decision.Engine (or a new type) to consume cre.AuthorizedFinding specifically. This is the single highest-leverage gap: without it, boundary F cannot be audited, attack 5 (forged finding) has no downstream target

to defend, and the reviewer’s own pipeline diagram has a literal hole at its center.
2. Build one real HTTP route in veriqo/gateway/rest that reaches the authority core (e.g. a route that submits evidence via insevidence.Registry.Submit, or queries a manifest.Manifest’s State). Only once this exists can boundary A be audited for real, and attack 12 (API injection) has a target.
3. Wire pkg/platform/audit (Ledger) to cite real authority-artifact hashes (ManifestHash, AuthorizationHash) as ledger-entry provenance, with the same "derive, never accept a caller-asserted hash" discipline the rest of this engagement has used everywhere else.
4. Extend ManifestAdapter/EvidenceAdapter onto the real pkg/transport/rafttcp harness (test/integration/live_cluster_test.go’s own pattern), closing boundary J’s remaining honest gap (real network, not MemTransport).
5. Decide and clarify "UCR"/"UER" against this repository’s actual package names (pkg/ucr is Unified Cognitive Reasoning, not a registry; no "UER" package exists) before auditing boundary C -- auditing an undefined boundary produces a false sense of coverage.
6. Wire pkg/storage/snapshot.Store (the generic, content-agnostic persisted-object store) to the authority core with the same replay-through-real-gates discipline raftlite.Snapshotter now uses, closing boundary H’s remaining gap -- these are two different mechanisms and only one is closed.
7. Once 1-2 above exist, build the actual end-to-end pipeline test (External Input -> API -> ... -> Reporting) and run the 12 named attacks against it for real -- this is the reviewer’s stated "most important test," and it is only meaningful once there is a real pipeline to attack.
8. Add the two small, currently-uncovered attack shapes (9: duplicate event specifically at the raftlite Apply-replay layer; 10: a general stale-read-then-act scenario) as targeted unit tests, independent of the larger roadmap items -- these are cheap and can be done any time.

VERIFICATION

No source code was changed for this document (this round’s other deliverable, the MLETR/eBL Conformance Mapping, is likewise a document, per the reviewer’s own "yang harus kita lakukan sekarang bukan coding lagi" -- what we need to do now is not coding anymore). The full repository test suite was re-run to confirm the codebase remains exactly as green as the prior round left it:

gofmt -l .	clean
go build ./...	clean
go vet ./...	clean
go test ./...	full repository suite: 189 packages, 0 FAIL

HONEST SCOPE BOUNDARY

- This document is a gap analysis and roadmap, not the executed OS Integration Audit itself. Executing it -- building the missing wiring in items 1-6 above, then running the real end-to-end adversarial pipeline -- is genuinely several more engineering rounds of work, not something this round overclaims to have finished.
- Every "not wired" finding above is a real, repository-wide grep result as of this round’s commit, not an assumption from a package’s name.
- Where this repository’s own package names did not cleanly match the reviewer’s terminology (specifically "UCR"/"UER"), that mismatch is reported plainly rather than papered over with a best-guess mapping.